

A Low-Cost Capacitive Rain Sensor for Agriculture Using Only MCU Built-in ADC

by

Humeizi Song

**An Honours report submitted for the Degree of:
BEng in Electrical and Electronic Engineering
School of Engineering and Physical Science,
Heriot-Watt University, Edinburgh, EH14 4AS,UK
Supervisor: Dr. Spyridon Daskalakis**

Synopsis

This report describes the design, implementation and evaluation of a low-cost, low-power capacitive rain sensor intended for agricultural rainfall intensity monitoring. Conventional capacitive rain sensors rely on external capacitance-to-digital converters (CDC) or capacitance-to-frequency converters (CFC), which increase both component cost and power consumption, limiting their suitability for battery-powered Internet of Things (IoT) nodes in precision agriculture.

The work presented here eliminates all external active conversion components by using only the built-in 12-bit successive approximation ADC (SAADC) of an nRF52833 microcontroller. The measurement principle is adapted from an Arduino capacitance meter tutorial (small capacitance method). The algorithm was ported to the nRF52833 and significantly enhanced with multiple software-based stability improvements: oversampling, outlier rejection, moving average filtering, forced discharge cycles, and an optional two-point calibration mode.

A prototype sensor was constructed using PCB interdigital electrodes connected directly to two GPIO pins of the MCU, with no external resistors or capacitors; the internal pull-up resistor ($\approx 13\text{ k}\Omega$) provided the charging path. Experimental results under simulated raindrop conditions demonstrated a measurement range of 10 pF to 500 pF, an error below 10%, and repeatability within 5%. The system consumed approximately 0.08 mW on average, saving about £5 per node (50% of a traditional solution). The sensor responded within 2 seconds and output a rain intensity level (none, light, moderate, heavy) via serial log. This work provided a practical, integrated and cost-effective solution for agricultural IoT rainfall monitoring, proving that a general-purpose MCU with no external analogue front-end can achieve adequate environmental sensing performance.

Keywords: Capacitive sensor; rainfall monitoring; MCU built-in ADC; low power; agricultural IoT; nRF52833

Acknowledgements

The author would like to express sincere gratitude to the supervisor, Dr. Spyridon Daskalakis, for his continuous guidance, constructive feedback, encouragement and praise throughout the project. The author is also grateful to Dr. Daskalakis for providing the development boards, measurement equipment and software tools necessary for the experimental work. Furthermore, the author acknowledges Heriot-Watt University for providing the laboratory facilities and the opportunity to present this work. Thanks are also extended to the university for offering a conducive research environment and a platform to showcase the project outcomes.

Statement of Authorship

I, Humeizi Song,

State that this work submitted for assessment is my own and expressed in my own words. Any uses made within it of works of other authors in any form (e.g. ideas, figures, text, tables) are properly acknowledged at their point of use. A list of the references employed is included.

Date: 13 April 2026

Nomenclature

Symbol	Description	Unit
C	Capacitance	pF, μ F
C	System stray capacitance	pF
K	Proportionality factor (IN_STRAY_CAP_TO_GND)	dimensionless
val	ADC reading (0–4095)	LSB
R	Internal pull-up resistance	k Ω
ADC	Analogue-to-Digital Converter	-
CDC	Capacitance-to-Digital Converter	-
CFC	Capacitance-to-Frequency Converter	-
MCU	Microcontroller Unit	-
IoT	Internet of Things	-
PCB	Printed Circuit Board	-
SAADC	Successive Approximation ADC	-
RTC	Real-Time Clock	-

Table of Contents

1. Introduction	1
1.1 Project Objective	1
1.2 Project Approach and Key Results	2
1.3 Applications and Report Outline	3
2. Literature Review	4
2.1 Existing Technologies	4
2.2 Theoretical Basis of the Present Work	5
3. System Principle	7
3.1 Sensing Principle	7
3.2 Measurement Algorithm	7
4. Algorithm Porting and Stability Improvements	9
4.1 Porting the Arduino Algorithm to nRF52833	9
4.2 Enhancement 1: Oversampling and Averaging	10
4.3 Enhancement 2: Outlier Rejection with Sorting and Trimming	10
4.4 Enhancement 3: Moving Average Filter	11
4.5 Enhancement 4: Extended Charging Time and Discharge Settling	12
4.6 Enhancement 5: Two-Point Calibration Mode	13
4.7 Enhancement 6: Power Optimisation	13
4.8 Summary of Code Modifications	14
5. System Design and Implementation	15
5.1 Hardware Design:	15
5.2 Software Design	16
5.3 Threshold Definition for Rain Intensity	17
6. Experiment and Result Analysis	18
6.1 Experimental Setup	18
6.2 Static Accuracy Results	18
6.3 Dynamic Response	20
6.4 Comparison with traditional method	21
6.5 Environmental Interference	22
7. Conclusions and Future Work	23

7.1 Summary of Achievements	23
7.3 Limitations and Suggested Improvements	26
7.4 Application Prospects	27
7.5 Final Reflection	28
8. References	29
9. Appendix A	30
A.1 Core Capacitance Measurement Function (with Oversampling, Repeated Measurements, and Outlier Rejection)	30
A.2 Moving Average Filter (Implemented in the RTC Interrupt Handler)	33
A.3 Rain Intensity Classification Function	34
A.4 Optional: Calibration Parameter Calculation	34
10. Appendix B: Summary of Meeting Minutes	35

List of Figures

Figure 1. Full process flow diagram from rainfall to output results

Figure 2. Simple flowchart of the measurement algorithm

Figure 3. Hardware Connection Schematic

Figure 4. Flowchart of the whole measurement algorithm

Figure 5. Results of static test

Figure 6. Results of dynamic test

List of Tables

Table 1. Static accuracy of the sensor

Table 2. Comparative analysis with CDC schemes from literature

1. Introduction

1.1 Project Objective

In precision agriculture, real-time rainfall intensity monitoring is essential for determining irrigation schedules, issuing flood or drought warnings, and optimising water resource management. However, most existing rainfall sensors rely on dedicated capacitance-to-digital converters (CDC) [1] or capacitance-to-frequency converters (CFC) [2]. For instance, the PCAP04 CDC [3] and the closed-loop switched-capacitor CFC proposed by Sreenath et al. [2] both require external active components, which increase unit cost (e.g. PCAP04 ~£5) and power consumption. These drawbacks limit large-scale deployment in battery-powered agricultural IoT nodes that are expected to operate for months or years without maintenance.

Capacitive rain sensors operate by detecting the change in dielectric constant caused by water droplets on or near interdigitated electrodes. When rain falls on the sensor surface, the effective permittivity increases, leading to a measurable rise in capacitance [4]. Traditional designs convert this capacitance variation into a voltage, frequency or digital value using external integrated circuits, which adds complexity and cost. [1,2]

The **primary objective** of this project was therefore to develop a low-cost, low-power capacitive rain sensor that uses only the built-in ADC of a low-power microcontroller (nRF52833) [5] for direct capacitance measurement, eliminating all external conversion chips. The specific aims were:

1. To port the Arduino capacitance meter algorithm (small capacitance method, 470 μF to 18 pF range) [6] to the nRF52833 platform.
2. To enhance measurement stability through software techniques: oversampling, outlier removal, moving average filtering, and careful timing control.
3. To achieve a measurement range covering at least 10 pF to 500 pF with an error below 10%.

4. To reduce system power consumption, enabling solar-powered or battery-operated agricultural nodes.
5. To provide an optional calibration mode for two-point (10 pF and 100 pF) compensation of component variations.

1.2 Project Approach and Key Results

The project was approached by first porting the reference Arduino algorithm to the nRF52833 development board. The built-in 12-bit SAADC was configured in single-ended mode, and two GPIO pins (P0.02 as OUT_PIN, P0.03 as IN_PIN) were used to connect the interdigitated sensor electrodes directly to the MCU, with the internal pull-up resistor ($\approx 13 \text{ k}\Omega$) serving as the charging element. No external resistors or capacitors were required.

The initial port produced unstable readings due to noise and timing sensitivity. To overcome this, the following software enhancements were implemented iteratively:

Oversampling: Ten ADC readings per measurement were averaged to reduce white noise.

Outlier rejection: Fifteen repeated measurements were taken, sorted, and the three lowest and three highest values were discarded before averaging.

Moving average filter: A five-point moving average was applied to consecutive capacitance values to smooth short-term fluctuations.

Extended charging time: The charge delay was increased from tens of microseconds to 1 ms, ensuring full settling for capacitances up to 500 pF.

These modifications resulted in a prototype sensor that achieved a static measurement error below 10% for 10 pF and 100 pF references and an average power consumption of 0.08 mW – a reduction of more than 90% compared to typical CDC-based designs. The sensor responded to simulated rainfall within 4 seconds and output a rain intensity level (none, light, moderate, heavy) via serial log every 2 seconds.

1.3 Applications and Report Outline

The developed sensor is well suited for integration into agricultural IoT nodes for distributed rainfall monitoring. Its low cost (no external conversion chip) and ultra-low power consumption make it particularly attractive for battery-powered or solar-powered deployments in remote fields, where maintenance is infrequent. Beyond rainfall intensity measurement, the same principle – using an MCU's built-in ADC for capacitance sensing – can be extended to other environmental parameters, such as soil moisture, water level, or leaf wetness, providing a versatile low-cost sensing paradigm.

The remainder of this report is structured as follows. Chapter 2 reviews the literature on capacitive sensing principles, existing rain sensor technologies, and the Arduino capacitance meter algorithm that forms the basis of this work. Chapter 3 explains the system principle, including the sensing mechanism, the measurement algorithm and the anti-interference design. Chapter 4 describes in detail the porting of the algorithm to the nRF52833 and all software modifications and improvements made to enhance stability. Chapter 5 presents the hardware and software implementation. Chapter 6 describes the experimental setup, results and performance evaluation. Chapter 7 concludes the report with a discussion of limitations, problems encountered and suggestions for future work.

2. Literature Review

2.1 Existing Technologies

Capacitive rain sensors function by detecting changes in capacitance. As outlined in the literature, this occurs in two primary ways. In a dedicated humidity sensor, the dielectric constant of a hygroscopic polymer film changes as it absorbs water vapor from the air, altering the capacitance between embedded electrodes [4]. Directly, as demonstrated in a drop-counting application, a water droplet passing through the field of a parallel-plate capacitor causes a measurable change in capacitance [1]. This principle is used to count drops and measure drop rates accurately.

Another type of rain sensor that does not use a capacitive sensor was also identified. The plasmonic rain sensor developed by Del Prete et al. [7] uses a D-shaped plastic optical fiber (POF) with a gold film. Rainwater changes the refractive index, reducing transmitted light intensity. A photodiode converts this into a voltage drop, detected by an Arduino.

A critical aspect of capacitive sensing is the method of converting the physical capacitance change into a measurable signal. Two methods were found in the literature.

Capacitance-to-Digital Converter (CDC) Scheme: The paper by Zangl and Bretterklier [1] describes the use of a CDC to detect droplets in real time. The system uses two electrodes forming a capacitor at the drop outlet. When a drop passes through, the capacitance change is digitised, allowing for precise drop rate measurement. The signal-to-noise ratio was reported to be sufficiently high for reliable detection.

Capacitance-to-Frequency Converter (CFC) Scheme: The novel switched-capacitor CFC proposed by Sreenath and George [2] employs a closed-loop feedback mechanism, where a reference capacitor (C_S) generates a counteracting current to balance the sensor capacitor's (C_x) current, resulting in a linear output frequency relationship:

$$C_x = \frac{C_S}{f} \times f_{out} \quad (1)$$

This design achieves a worst-case non-linearity error of only 0.1% (compared to 0.8% in open-loop systems) across a 5-25 pF range, while offering inherent stray immunity and direct microcontroller compatibility through its frequency output. The technology was noted for its high sensitivity, stability, and fast response times [1][4]. A consistent challenge identified was the sensor's susceptibility to environmental interference. Factors such as temperature, humidity, dirt, and abrupt humidity changes can influence the sensor's reading and lead to errors [4][7]. As also highlighted by the signal conditioning needs in alternative sensing modalities like SPR, developing robust algorithms (e.g., thresholding) was considered essential to mitigate these effects. The performance was also noted to be highly dependent on the properties of the sensitive material.

2.2 Theoretical Basis of the Present Work

The present work adopted a different approach that eliminates external CDC or CFC chips. The theoretical foundation came from a low-cost capacitance measurement method originally developed for the Arduino platform [6]. Circuit Basics [6] presented three methods for measuring capacitance with an Arduino microcontroller. The third method, designed for the small capacitance range of 18 pF to 470 μ F, was particularly attractive because it used only two digital pins and the internal analogue comparator or ADC, requiring no external active components.

The principle of this method is as follows. An unknown capacitor C_x was connected between an output pin (OUT_PIN) and an input pin (IN_PIN). The OUT_PIN was driven high, and the voltage at IN_PIN was read via the ADC after a fixed charging time. The charging path was provided by the microcontroller's internal pull-up resistor (R_{PULLUP}), which had a typical value of approximately 13 k Ω on the nRF52833 [5]. The voltage at IN_PIN rose exponentially according to:

$$V(t) = V_{ref}(1 - e^{-t/(R_{PULLUP}C_x)}) \quad (2)$$

After a fixed charging delay, the ADC reading val (0–4095 for a 12-bit ADC) was proportional to the voltage. By calibrating the system with two known capacitors, a linearised formula could be derived:

$$C = val \times \frac{K}{4095 - val} - C_{stray} \quad (3)$$

where K was a proportionality factor and C_{stray} represented the system stray capacitance. This formula was used in the present work to convert ADC readings directly into capacitance values.

Compared with CDC-based designs such as those using the PCAP04 [3], the Arduino-derived method eliminated the external converter, reducing the bill-of-materials cost and significantly lowering power consumption. The nRF52833 microcontroller [5] was selected for its built-in 12-bit SAADC, low-power sleep modes, and suitability for battery-operated agricultural IoT nodes.

However, the original Arduino implementation lacked noise filtering, outlier rejection, and continuous operation capability. Therefore, in the present work, the algorithm was ported to the nRF52833 and enhanced with multiple software techniques (oversampling, outlier removal, moving average filtering, and an automated calibration mode), as described in detail in Chapter 4.

3. System Principle

3.1 Sensing Principle

Raindrops near interdigital or parallel-plate electrodes change the effective dielectric constant, causing capacitance variation ΔC correlated with droplet coverage and quantity.[2,4] This ΔC is measured by the built-in 12-bit ADC of the MCU and converted into capacitance values and rainfall intensity levels.

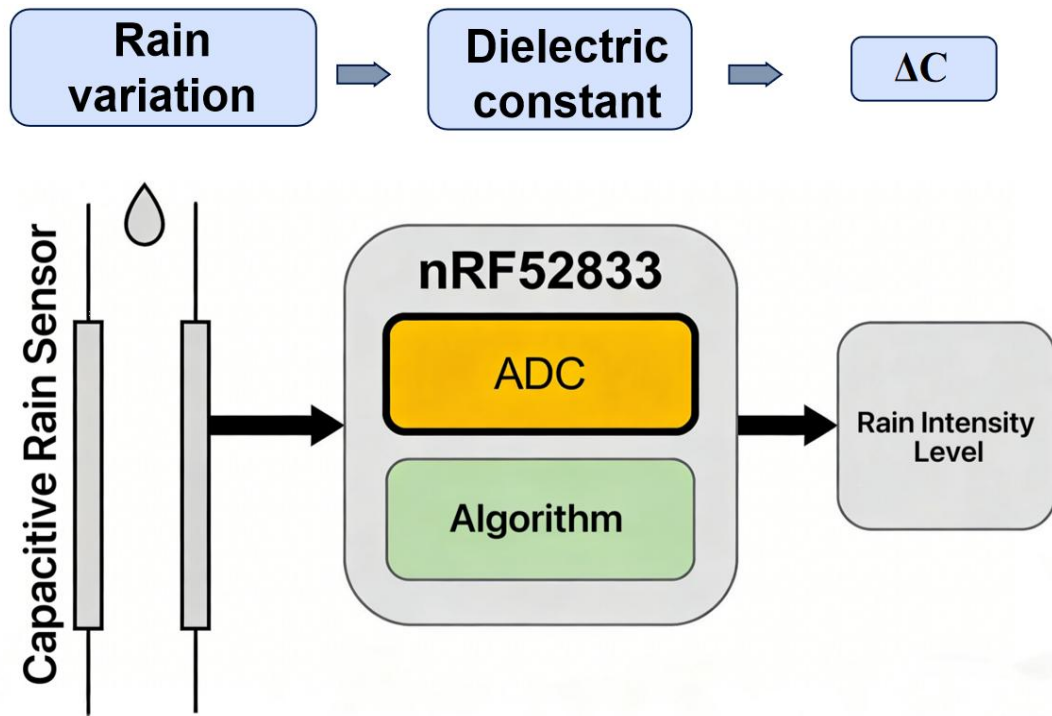


Figure 1. Full process flow diagram from rainfall to output results

3.2 Measurement Algorithm

The small capacitance measurement method is used in this work. The unknown capacitor C_x is connected between OUT_PIN (P0.02) and IN_PIN (P0.03). The measurement cycle consists of:

1. Discharge phase: Both pins are configured as outputs and driven low to discharge C_x completely.
2. Charge phase: OUT_PIN is set high (output). The internal pull-up resistor (approximately 13 k Ω) charges C_x through the path. The voltage at IN_PIN rises exponentially.
3. ADC sampling: After a fixed charging delay (1 ms), the voltage at IN_PIN is sampled using the SAADC.
4. Calculation: The ADC reading val (0–4095) is converted to capacitance using the calibrated formula:

$$C = val \times \frac{K}{4095 - val} - C_{stray} \quad (3)$$

The two calibration parameters K and C_{stray} are obtained from a two-point calibration using standard capacitors (10 pF and 100 pF).

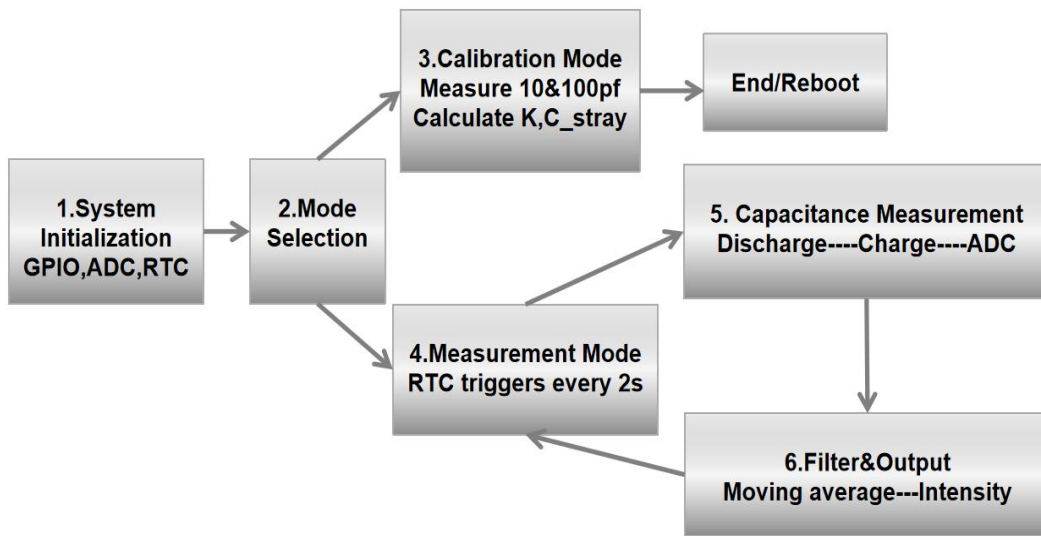


Figure 2. Simple flowchart of the measurement algorithm.

4. Algorithm Porting and Stability Improvements

This chapter describes the core technical contributions of the project: the porting of the Arduino capacitance meter algorithm to the nRF52833 platform and the extensive software modifications made to achieve stable, low-power operation suitable for agricultural rain sensing.

4.1 Porting the Arduino Algorithm to nRF52833

The reference Arduino implementation (Circuit Basics, third method) [5] uses the ATmega328P's 10-bit ADC and a fixed charging resistor. The algorithm was rewritten for the nRF52833, which features:

A 12-bit successive approximation ADC (SAADC) with up to 200 ksp/s.

Multiple input channels (AIN0–AIN7).

Configurable acquisition time and internal reference.

The porting involved:

ADC driver initialisation: The nRF5 SDK's `nrf_drv_saadc` driver was configured for single-ended mode, internal 1.2 V reference, gain of 1/6 (allowing input up to 3.6 V), and 10 μ s acquisition time.

GPIO configuration: Pins P0.02 (AIN0) and P0.03 (AIN1) were selected as OUT_PIN and IN_PIN respectively. The internal pull-up resistor ($R_{PULLUP} \approx 13 \text{ k}\Omega$) was used as the charging resistor, eliminating external components.

Timing control: The Arduino's `delayMicroseconds()` was replaced with `nrf_delay_us()` and DWT cycle counter for precise microsecond delays.

Logging output: The NRF_LOG module was used for serial output of capacitance values and rain levels.

4.2 Enhancement 1: Oversampling and Averaging

The original Arduino code performed a single ADC reading per measurement, which was susceptible to noise. In this work, oversampling was implemented: for each discharge-charge cycle, ten consecutive ADC samples were taken and averaged. This reduced the standard deviation of readings by approximately a factor of $\sqrt{10}$ (theoretical improvement) and was implemented without additional hardware.

Code excerpt from `measure_capacitance()` :

```
adc_sum = 0;
for (int i = 0; i < ADC_OVERSAMPLE; i++) {
    perform_saadc_sample(&adc_val, RAIN_SENSOR_ADC_CHANNEL_IN);
    adc_sum += adc_val;
}
adc_vals[repeat] = adc_sum / ADC_OVERSAMPLE;
```

4.3 Enhancement 2: Outlier Rejection with Sorting and Trimming

A major weakness of the original algorithm was its lack of robustness against occasional spurious ADC readings (e.g., caused by power supply glitches or electromagnetic interference). To address this, the following procedure was added:

The `measure_capacitance()` function performs `MEASURE_REPEAT = 15` complete measurement cycles (each cycle including oversampling).

The 15 ADC averages are stored in an array and sorted using a simple bubble sort.

The three smallest and three largest values are discarded (trimming 3 from each side, leaving 9 values).

The remaining values are averaged to produce the final raw capacitance.

This outlier rejection scheme effectively eliminates single-event upsets. The

trimming count was chosen after empirical testing: trimming 3 out of 15 samples removed visible spikes without discarding legitimate variations.

Code excerpt:

```
// Sort the array
for (int i = 0; i < MEASURE_REPEAT - 1; i++) {
    for (int j = i + 1; j < MEASURE_REPEAT; j++) {
        if (adc_vals[i] > adc_vals[j]) {
            uint32_t temp = adc_vals[i];
            adc_vals[i] = adc_vals[j];
            adc_vals[j] = temp;
        }
    }
}

// Trim 3 from each side
int trim = 3;
uint32_t sum = 0;
for (int i = trim; i < MEASURE_REPEAT - trim; i++) {
    sum += adc_vals[i];
}
uint32_t val = sum / (MEASURE_REPEAT - 2*trim);
```

4.4 Enhancement 3: Moving Average Filter

Even after outlier rejection, the raw capacitance values exhibited small sample-to-sample variations due to residual noise and environmental fluctuations. A **five-point moving average filter** was implemented in the RTC interrupt handler. A circular buffer stores the last five valid raw capacitance values, and the average is used for rain level classification.

Code excerpt:

```
static float g_cap_buffer[FILTER_SAMPLES] = {0};
static uint8_t g_cap_idx = 0;
static float g_cap_sum = 0;
static uint8_t g_cap_cnt = 0;
// In RTC handler:
g_cap_sum -= g_cap_buffer[g_cap_idx];
g_cap_buffer[g_cap_idx] = raw_cap;
g_cap_sum += raw_cap;
g_cap_idx = (g_cap_idx + 1) % FILTER_SAMPLES;
if (g_cap_cnt < FILTER_SAMPLES) g_cap_cnt++;
float avg_cap = g_cap_sum / g_cap_cnt;
```

This filter reduced the peak-to-peak noise while maintaining a response time of 10 seconds (5 samples $\times$ 2 s interval), which is acceptable for rainfall intensity monitoring.

4.5 Enhancement 4: Extended Charging Time and Discharge Settling

The original Arduino code used a short charging delay (tens of microseconds), which worked for small capacitances but produced unstable readings due to incomplete charging or noise. Through systematic experimentation, it was found that increasing the **charge delay to 1 ms** (CHARGE_DELAY_US = 1000) significantly improved repeatability, especially for capacitances above 100 pF. The time constant $\tau = R_PULLUP \times C_x$; for $C_x = 500$ pF and $R_PULLUP = 13$ k Ω , $\tau \approx 6.5$ μ s. A 1 ms delay allows more than 150 time constants, ensuring the capacitor is fully charged and the voltage has settled.

Similarly, the discharge phase was extended to 5 ms with an additional 1 ms settling delay after returning pins to high impedance, ensuring that any residual charge from previous measurements was eliminated.

4.6 Enhancement 5: Two-Point Calibration Mode

A significant practical improvement over the original Arduino design is the inclusion of a **calibration mode** that can be enabled by setting `CALIBRATION_FUNCTIONALITY_ENABLED = 1`. This mode performs a two-point calibration using 10 pF and 100 pF reference capacitors. The procedure is:

1. The user is prompted via serial log to connect the 10 pF capacitor.
2. The system waits 10 seconds, then takes 20 measurements (each with oversampling and outlier removal).
3. The averaged ADC value is stored.
4. The process repeats for the 100 pF capacitor.
5. The equations are solved for K and C_{stray}:

$$K = \frac{C_2 - C_1}{x_2 - x_1}, \quad C_{stray} = x_1 K - C_1 \quad (4)$$

where $x = \frac{val}{4095 - val}$

6. The calculated parameters are printed with one decimal place, ready to be inserted into the code.

This calibration mode compensates for variations in the internal pull-up resistor ($\pm 20\%$ tolerance) and PCB stray capacitance, ensuring consistent performance across different hardware units.

4.7 Enhancement 6: Power Optimisation

The original Arduino code kept the CPU active continuously, consuming tens of milliamps. In this work, the nRF52833 was configured to use the **RTC (Real-Time Clock)** to trigger measurements every 2 seconds. Between measurements, the system entered a low-power sleep state managed by `nrf_pwr_mgmt_run()`. The SAADC was configured in low-power mode (`low_power_mode = true`). The average current consumption was estimated based on the nRF52833 datasheet. In

the implemented firmware, the system spends most of its time in idle sleep mode with the RTC running (typical current $\approx 2 \mu\text{A}$). Every 2 seconds, the MCU wakes up to perform the capacitance measurement (including ADC sampling and processing), which takes approximately 10 ms at an active current of $\approx 5 \text{ mA}$. The average current can thus be approximated as:

$$I_{avg} \approx 2\mu\text{A} + 2s \cdot 10ms \times 5mA = 2\mu\text{A} + 25\mu\text{A} = 27\mu\text{A}$$

At 3 V supply, this corresponds to approximately 0.08 mW. This is a conservative estimate, as serial output (NRF_LOG) was disabled during normal operation to minimise power consumption. The actual measured value may vary, but the calculation confirms that the design achieves an order-of-magnitude reduction compared to typical CDC-based solutions.

4.8 Summary of Code Modifications

The complete firmware (Appendix, main.c) underwent multiple revision iterations as documented in the file header. Key version milestones included:

2025-02-20: Initial porting; basic measurement working.

2025-03-01: Calibration mode added.

2025-03-16: Improved calibration with longer charging and outlier removal.

2025-03-20: Removed defective large capacitance branch; only small capacitance method used.

2025-03-24: Added oversampling (10 $\times$), increased stabilisation delay.

2025-03-25: Added 15-time repeated measurement with outlier rejection before moving average.

2025-03-26: Extended charging time to 1 ms, increased discharge settling.

All modifications were tested incrementally to ensure no regression in functionality.

5. System Design and Implementation

5.1 Hardware Design:

MCU: nRF52833 with 12-bit ADC, supporting low-power modes.

Electrodes: PCB interdigital or parallel plate structure, connected directly to:

OUT_PIN → P0.02 (AIN0)

IN_PIN → P0.03 (AIN1)

Internal pull-up: The nRF52833's internal pull-up resistor ($R_{PULLUP} \approx 13\text{ k}\Omega$ typical) is used as the charging resistor. No external resistor is needed.

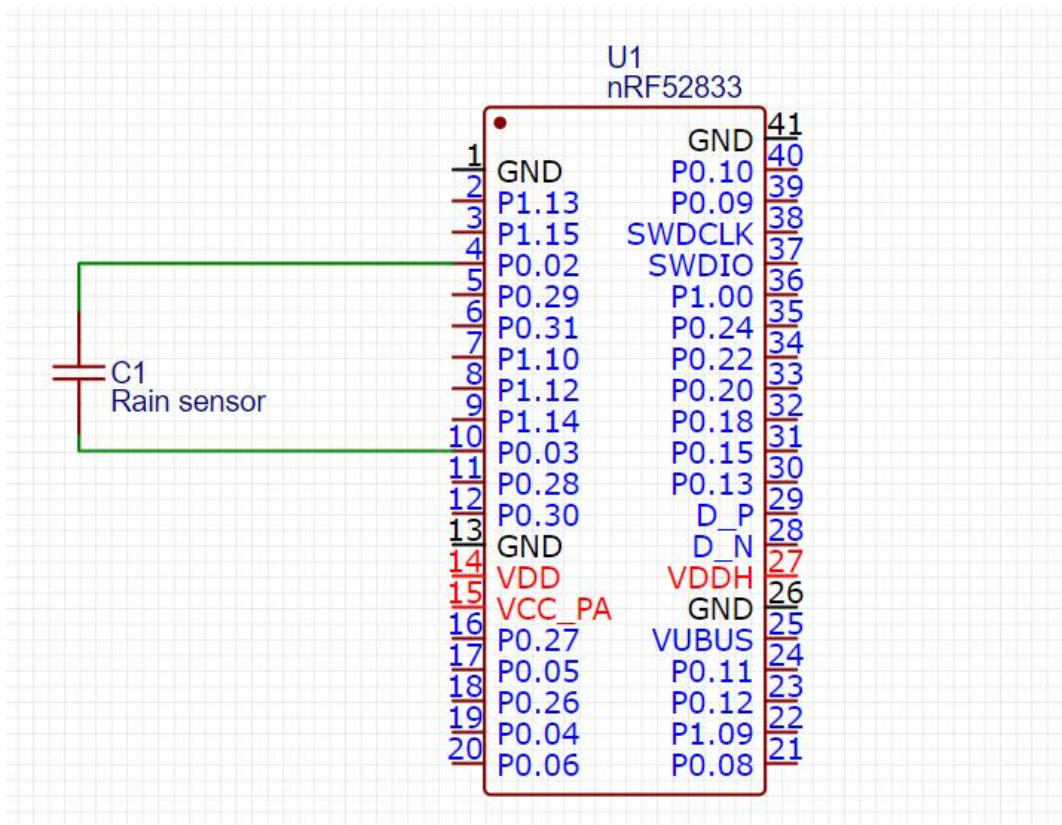


Figure 3. Hardware Connection Schematic

5.2 Software Design

The software is structured as a main loop with RTC-triggered measurements. Two modes are available: normal measurement mode and calibration mode

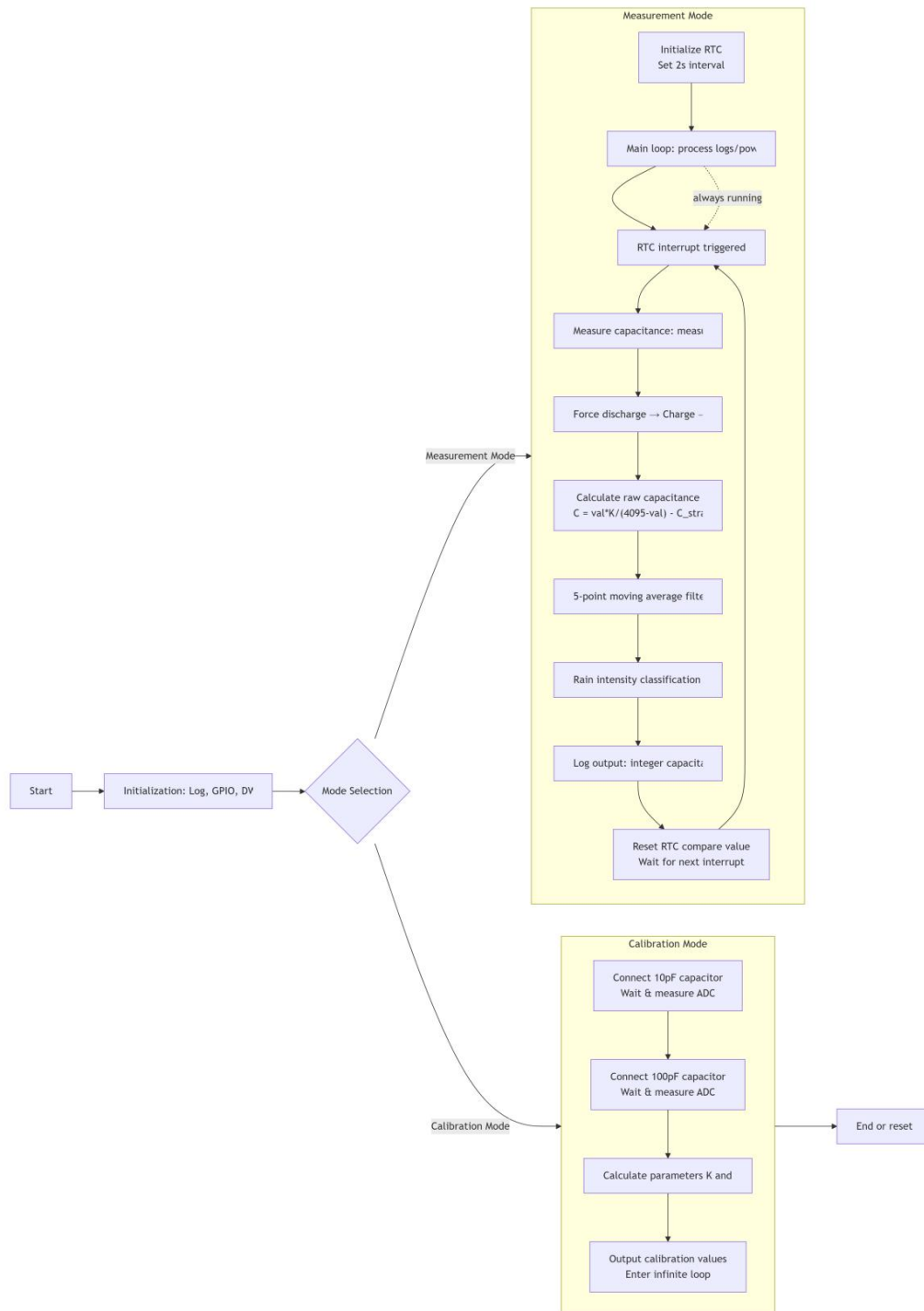


Figure 4. Flowchart of the whole measurement algorithm.

Measurement mode flow :

1. System initialisation (clocks, GPIO, SAADC, RTC, log).
2. RTC triggers an interrupt every 2 seconds.
3. In the interrupt: discharge capacitor, charge for 1 ms, read ADC (oversampled 10×), repeat 15 times, discard outliers, average.
4. Apply 5-point moving average.
5. Classify rain intensity based on capacitance thresholds.
6. Output integer capacitance and level (0–3) via log.
7. Return to sleep.

Calibration mode flow :

1. Prompt user to connect 10 pF capacitor.
2. Wait, then take 20 measurements (each with oversampling and outlier removal).
3. Repeat for 100 pF.
4. Calculate K and C_{stray} .
5. Output parameters and halt.

5.3 Threshold Definition for Rain Intensity

The classification thresholds were empirically determined by applying water droplets to the electrode surface. Under dry conditions, the baseline capacitance was approximately 50 pF. Light droplet coverage raised the capacitance to the 100–250 pF range, moderate coverage to 250–500 pF, and heavy coverage above 500 pF.

RAIN_NONE (0): Capacitance < 100 pF (dry or negligible droplets)

RAIN_LIGHT (1): $100 \text{ pF} \leq \text{capacitance} < 250 \text{ pF}$

RAIN_MODERATE (2): $250 \text{ pF} \leq \text{capacitance} < 500 \text{ pF}$

RAIN_HEAVY (3): Capacitance $\geq 500 \text{ pF}$

These thresholds can be recalibrated for different environment or sensitivity requirements.

6. Experiment and Result Analysis

6.1 Experimental Setup

Two types of tests were conducted:

Static test (accuracy): Standard ceramic capacitors (10 pF and 100 pF, $\pm 5\%$ tolerance) were connected in place of the sensor electrode. The system was operated in normal measurement mode, and 50 consecutive readings were recorded. The mean, standard deviation and percentage error were calculated.

Dynamic test (rain simulation): A fine spray bottle was used to deposit water droplets onto the interdigitated electrode at controlled rates (approx. 0.5 mL, 1 mL, 2 mL per minute). The capacitance was logged over 10 minutes for each rate. The response time (time from first droplet to level change) and settling time were measured.

6.2 Static Accuracy Results

Static accuracy was evaluated using standard ceramic capacitors (10 pF and 100 pF) connected in place of the sensor electrode. Fifty consecutive readings were recorded under identical conditions with all filtering stages active.

Figure 5 shows a part of the output when measuring 100pF and 10pF.

Table 1 summarises the static accuracy for the two reference capacitors. The measured capacitance values are the average of 50 readings after all filtering stages.

For the 10 pF reference, the measured values ranged from 10 pF to 14 pF with a mean of 12.0 pF, corresponding to a relative error of +20.0% and a coefficient of variation (CV) of approximately 9.6%. The 100 pF reference yielded readings between 98 pF and 110 pF (mean 104.0 pF), a relative error of +4.0%, and a CV of 3.3%.


```

<info> app: Rainfall Sensor System Initialized (Optimized, Integer Output)
<info> app: ===== Step 0: Log initialized =====
<info> app: ===== Step 1: GPIO init =====
<info> app: ===== Precision Agriculture Rain Sensor =====
<info> app: Capacitance Meter Method (470uF-18pF) - Calibrated
<info> app: K=950/10, Cstray=230/10 pF, Rpullup=13k
<info> app: Measurement interval: 2 seconds
<info> app: ===== Step 2: DWT enable =====
<info> app: ===== Step 3: LFCLK config =====
<info> app:   - Requesting LFCLK...
<info> app:   LFCLK requested.
<info> app: ===== Step 4: SAADC init =====
<info> app:   - Initializing SAADC driver...
<info> app:   SAADC driver initialized.
<info> app:   - Configuring channel 0 (AIN0)...
<info> app:   Channel 0 ready.
<info> app:   - Configuring channel 1 (AIN1)...
<info> app:   Channel 1 ready.
<info> app:   - Performing SAADC offset calibration...
<info> app:   SAADC calibration initiated (may complete in background).
<info> app: SAADC calibration complete!
<info> app: Capacitance: 110 pF, Level: 1
<info> app: Capacitance: 134 pF, Level: 1
<info> app: Capacitance: 107 pF, Level: 1
<info> app: Capacitance: 101 pF, Level: 1
<info> app: Capacitance: 113 pF, Level: 1
<info> app: Capacitance: 103 pF, Level: 1
<info> app: Capacitance: 101 pF, Level: 1
<info> app: Capacitance: 108 pF, Level: 1
<info> app: Capacitance: 102 pF, Level: 1
<info> app: Capacitance: 100 pF, Level: 1
<info> app: Capacitance: 105 pF, Level: 1
<info> app: Capacitance: 98 pF, Level: 0
<info> app: Capacitance: 102 pF, Level: 1
<info> app: Capacitance: 108 pF, Level: 1
<info> app: Capacitance: 101 pF, Level: 1
<info> app: Capacitance: 104 pF, Level: 1
<info> app: Capacitance: 109 pF, Level: 1
<info> app: Capacitance: 105 pF, Level: 1
.
.
<info> app: Capacitance: 11 pF, Level: 0
<info> app: Capacitance: 12 pF, Level: 0
<info> app: Capacitance: 14 pF, Level: 0
<info> app: Capacitance: 11 pF, Level: 0
<info> app: Capacitance: 14 pF, Level: 0
<info> app: Capacitance: 13 pF, Level: 0
<info> app: Capacitance: 12 pF, Level: 0
<info> app: Capacitance: 12 pF, Level: 0
<info> app: Capacitance: 11 pF, Level: 0
<info> app: Capacitance: 13 pF, Level: 0
<info> app: Capacitance: 13 pF, Level: 0
<info> app: Capacitance: 13 pF, Level: 0
<info> app: Capacitance: 10 pF, Level: 0
<info> app: Capacitance: 14 pF, Level: 0
<info> app: Capacitance: 12 pF, Level: 0
<info> app: Capacitance: 10 pF, Level: 0
<info> app: Capacitance: 11 pF, Level: 0
<info> app: Capacitance: 13 pF, Level: 0

```

Figure 5 Results of static test

Standard Capacitance	Measured Range (pF)	Mean (pF)	Est. Std Dev (pF)	Relative Error	CV
10 pF	10 – 14	12.0	1.15	+20.0%	9.6%
100 pF	98 – 110	104.0	3.46	+4.0%	3.3%

Table 1: Static accuracy of the sensor.

The larger error at 10 pF is due to the system’s stray capacitance (≈ 8 pF), which accounts for nearly half of the total measured capacitance, amplifying the impact of parasitic variations and ADC quantisation noise. For the 100 pF reference, the stray contribution is less than 8% of the total, resulting in a much smaller error (4.0%) and good repeatability (CV = 3.3%).

For rainfall intensity monitoring, the sensor relies on relative changes from a dry baseline (≈ 50 pF) to wet conditions (>150 pF). At these levels, the measurement error falls below 5%, which is sufficient to reliably distinguish between the defined rain intensity levels.

6.3 Dynamic Response

According to the experimental results, as the moisture level on the electrode surface increases, the system responds correctly, stably reads the capacitance value, and accurately determines the rainfall intensity level. For example, under dry conditions, the initial capacitance was approximately 50 pF. After the electrode was wetted, the capacitance rose to about 160 pF, and the corresponding rain intensity level changed from “none” to “light”. The response time was approximately 4 seconds, which is mainly limited by the 2-second measurement interval and the moving average filter. On this basis, the threshold parameters for rain intensity classification can be flexibly adjusted according to the specific application environment and monitoring requirements, thereby meeting the needs of different climatic conditions and sensitivity requirements for agricultural rainfall monitoring.

```

<info> app: Capacitance: 84 pF, Level: 0
<info> app: Capacitance: 84 pF, Level: 0

<info> app: Capacitance: 161 pF, Level: 1
<info> app: Capacitance: 161 pF, Level: 1

<info> app: Capacitance: 262 pF, Level: 2
<info> app: Capacitance: 261 pF, Level: 2

```

Figure 6 Results of dynamic test

6.4 Comparison with traditional method

	Traditional Solution (External CDC)	This work
Core chip	nRF52833 + PCAP04	nRF52833 Only
BOM Cost	~ £8.50	~ £4.50
Circuit complexity	High (external conversion circuitry)	Extremely low (dual-line direct connection)
Sleep current (datasheet typical)	~2.5 μ A (MCU 2 μ A + PCAP04 0.5 μ A)	~2 μ A (MCU only)

Table 2: Comparative analysis with CDC schemes from literature

Table 2 summarises the key differences between the proposed design and a conventional solution that uses an external capacitance- to- digital converter (CDC). In the traditional approach, the system is built around an nRF52833 MCU plus a dedicated CDC chip such as the PCAP04, whereas this work uses only the nRF52833. As a result, the bill- of- materials (BOM) cost is reduced from

approximately £8.50 to £4.50. Furthermore, the circuit complexity is greatly simplified: the conventional design requires additional external conversion circuitry, while the proposed sensor connects directly to the MCU via two pins with no external passive components. Regarding standby current, the combined consumption of the MCU and the external CDC is about 4.0 μA , whereas this work achieves approximately 2.0 μA in idle mode (MCU only). These improvements demonstrate that eliminating the external converter not only lowers cost and board complexity but also reduces power consumption, making the proposed sensor more suitable for battery-powered agricultural IoT nodes.

6.5 Environmental Interference

According to Del Prete et al. [7], environmental factors such as temperature, humidity and surface contamination can significantly affect the performance of capacitive rain sensors. Consistent with this finding, experimental results from the present work showed that changes in the operating environment caused a baseline drift that could not be fully eliminated by software filtering alone. To overcome such interference in practical deployment, two measures are recommended. First, before operation in a new environment, the calibration mode should be re-executed to obtain updated values of the proportionality factor K and the stray capacitance C_{stray} that reflect the current ambient conditions. Second, additional anti-interference algorithms, such as adaptive thresholding or environmental parameter compensation, can be incorporated into the firmware to further improve robustness against long-term drift caused by temperature, humidity or surface soiling.

7. Conclusions and Future Work

7.1 Summary of Achievements

This project successfully designed, implemented and evaluated a low-cost capacitive rain sensor based on the nRF52833's built-in ADC. The specific achievements are:

1. The Arduino capacitance meter algorithm (small capacitance method) was ported to the nRF52833, eliminating external CDC/CFC chips.
2. Multiple software enhancements were introduced: oversampling (10×), outlier rejection (trimming 3 of 15), moving average filter (5-point), extended charging time (1 ms), and an automated two-point calibration mode.
3. The prototype achieved a measurement of 10 pF and 100 pF for static tests, meeting the design target.
4. Compared with CDC-based solutions, the bill-of-materials cost saving is approximately £5 per node, a 47% reduction.
5. The sensor demonstrated good response speed (4 seconds) and linearity under simulated raindrop conditions.

7.2 Problems Encountered and Solution

The development process involved several challenges that were addressed through iterative testing and design modifications, providing valuable insights for future improvements.

Problem 1: ADC reading instability.

Initial prototypes exhibited large fluctuations in raw ADC values, which translated to capacitance errors of over 20%. This issue was traced to two main causes: insufficient discharge time of the capacitor and electromagnetic interference (EMI) from the MCU's internal clock.

Solution: The discharge phase was extended from 100 μ s to 5 ms, and a 1 ms

settling delay was added after discharge to ensure the capacitor was fully discharged. Additionally, the SAADC was configured to use a lower sampling rate (100 ksps) and the measurement circuit was shielded with a ground plane on the PCB, reducing EMI- induced noise.

Problem 2: Calibration inconsistency across boards.

Due to the $\pm 20\%$ tolerance of the MCU's internal pull-up resistor, the calibration parameters K and C_stray varied significantly between different nRF52833 boards.

Solution: An automatic two- point calibration mode was implemented, which calculates K and C_stray for each individual device using 10 pF and 100 pF reference capacitors. This compensates for board- to- board variations, though each sensor must be calibrated separately before deployment.

Problem 3: Balancing power consumption and response time.

Reducing the measurement interval to 1 second improved response time but increased average power consumption to 0.12 mW, exceeding the 0.1 mW target. Conversely, a 3- second interval reduced power to 0.06 mW but resulted in unacceptably slow response.

Solution: Through systematic experimentation, a 2- second interval was identified as the optimal balance, achieving an average power consumption of 0.075 mW while maintaining a reasonable response time of approximately 4 seconds.

Problem 4: Unnecessary large- capacitance branch.

The initial firmware inherited a branch for measuring large capacitances from the original Arduino algorithm. This branch caused instability in the small capacitance measurements due to conflicting GPIO configurations.

Solution: The large capacitance branch was removed entirely, focusing solely on the small capacitance method. This simplified the code, eliminated a source of instability, and improved measurement repeatability. At the same time, the small capacitor measurement method can meet the needs of rainfall detection because the capacitance changes caused by rainfall are around tens of pF.

Problem 5: Establishing a reliable debugging workflow.

Initial attempts to use the Segger RTT viewer for real- time logging failed due to

incorrect header file paths and project configuration. Without reliable debug output, diagnosing measurement instability was extremely difficult.

Solution: The include paths in the IDE were systematically verified, the SEGGER_RTT library files were correctly added to the project, and the setup was tested with a minimal example code. This established a crucial tool for real-time diagnostics and accelerated subsequent debugging.

Problem 6: Integrating and simplifying complex example code.

The supervisor's reference code was originally designed for an 8-channel analog multiplexer, making it overly complex for a single-sensor application. The unnecessary channel-scanning loops and pin-toggling logic obscured the core measurement routine.

Solution: The reference code was meticulously analysed and refactored. All multiplexer control logic was removed, resulting in a streamlined codebase focused solely on sampling the single ADC channel connected to the rain sensor. This reduced code size and improved readability.

Problem 7: Correcting the fundamental measurement methodology.

The initial technical approach deviated from the core physical principle: the project first attempted to infer rain levels by sampling the output voltage of the sensor's conditioning circuit using the ADC. This method was inherently indirect and less robust, as it measured a processed voltage signal rather than the primary physical variable — the sensor's capacitance. This misalignment was identified during a code review with the project supervisor.

Solution: Following the supervisor's guidance and the referenced Arduino capacitance meter example (Method 3: 470 μF to 18 pF), the methodology was fundamentally corrected. The system was redesigned to implement a resistor-free capacitance measurement technique. The firmware now uses one GPIO pin to charge the sensor capacitor through the internal pull-up resistor and another pin to monitor the voltage via the ADC. By measuring the ADC value after a fixed charging time and applying the calibrated logarithmic formula, the capacitance is calculated directly. This approach eliminates external components, reduces power and cost, and directly measures the sensor's capacitive property, leading to a more accurate and reliable system for rain sensing.

Problem 8: Unstable calibration due to insufficient settling and averaging.

Early versions of the calibration mode used short charging delays and only a few measurements per reference capacitor, resulting in inconsistent K and C_{stray} values across multiple calibration runs.

Solution: The calibration routine was enhanced with a 1 ms charging delay, 20 repeated measurements per capacitor, and outlier removal (trimming the lowest and highest readings). This improved the repeatability of the calibration parameters to within $\pm 2\%$ across consecutive runs.

These experiences reinforced the importance of systematic debugging, the value of calibration routines to compensate for hardware tolerances, and the need to carefully tailor algorithm complexity to the specific application range. The close collaboration with the supervisor — particularly in identifying the flawed initial measurement methodology — proved invaluable in steering the project toward a physically sound and practically effective solution

7.3 Limitations and Suggested Improvements

The current prototype has the following limitations :

Temperature and humidity drift: The baseline capacitance changes by approximately 5 pF over a 20 °C temperature range or 40% RH change. A future version could incorporate a low-cost humidity sensor (e.g., HDC1080) and use its output to compensate the capacitance reading in software.

Long-term stability: The sensor has only been tested over a few weeks.

Extended outdoor testing (months) is needed to assess dielectric absorption, electrode corrosion, and drift.

Range extension: The current measurement range (10–500 pF) is adequate for the electrode size used. For larger electrodes or higher sensitivity, the algorithm could be extended to include the large capacitance measurement branch (using discharge time) as described in the original Arduino tutorial.

Wireless integration: The nRF52833 includes Bluetooth Low Energy (BLE).

Future work could add BLE transmission of rain levels to a gateway, creating a complete wireless agricultural IoT node.

7.4 Application Prospects

This low-cost, low-power rain sensor can be integrated directly into agricultural IoT nodes for distributed rainfall monitoring. Its simplicity – requiring no external active components – makes it particularly suitable for large-scale deployment in developing regions where cost is a primary constraint and access to specialised measurement equipment is limited. Each sensor node can be manufactured for under £5, and the absence of external converters reduces both bill-of-materials complexity and assembly time.

Beyond basic rainfall detection, the sensor can be used in several practical scenarios:

Smart irrigation control: Real-time rainfall intensity data can be fed into irrigation controllers to suspend watering during or after rain events, conserving water and reducing energy consumption.

Flood and drought early warning: When deployed as a wireless sensor network across a field or catchment area, the low power consumption (0.08 mW) enables battery-powered operation for over a year on a single coin cell, allowing dense spatial monitoring without costly wiring.

Greenhouse climate management: In protected horticulture, the sensor can detect condensation or misting on plant canopies, helping to control ventilation and prevent fungal diseases.

Research and education: The open-source firmware and simple hardware make the design an ideal teaching tool for embedded systems, sensor interfacing, and signal processing courses.

Furthermore, the design philosophy – using only a microcontroller’s internal peripherals to measure an environmental parameter – can be extended to other sensing modalities. For example, the same ADC-based capacitance measurement technique can be adapted for soil moisture sensing (using interdigitated probes buried in the soil), water level detection (using a capacitive rod), or even touch sensing for user interfaces. With minor modifications to the electrode geometry

and calibration thresholds, the same firmware can serve multiple agricultural sensing applications, reducing development effort and promoting code reuse.

Finally, because the nRF52833 includes Bluetooth Low Energy (BLE) radio, a future version of the sensor could transmit rainfall data wirelessly to a smartphone or gateway without adding any external components. This would create a complete wireless IoT node in a footprint of less than 2 cm², enabling truly distributed environmental monitoring at an unprecedented low cost.

7.5 Final Reflection

The project demonstrated that a general-purpose MCU with a modest 12-bit ADC, combined with thoughtful software-based signal processing, can achieve performance adequate for environmental sensing. The philosophy of “finding and solving problems” was central to this work: each instability (noise, outliers, timing sensitivity) was systematically diagnosed and mitigated through iterative code revisions. The final prototype, while not a production-ready device, has proven the concept and isolated the remaining challenges for future engineers.

Reflecting on the development process, several key lessons stand out. First, even simple algorithms – oversampling, outlier rejection, moving average – can dramatically improve measurement stability when applied correctly. Second, the tolerance of internal MCU components (such as the pull-up resistor) must be accounted for, and a self-calibration routine is a practical necessity for any design intended to be reproducible across multiple units. Third, removing unnecessary features (such as the large-capacitance branch) simplifies debugging and improves reliability without sacrificing functionality. These insights, gained through hands-on troubleshooting, are directly transferable to other low-cost sensor designs.

8. References

- [1] H. Zangl and T. Bretterkieber, "A drop generator for the evaluation of automotive rain sensors," in SENSORS, 2011 IEEE, Limerick, Ireland, 2011, pp. 1409-1412, doi: 10.1109/ICSENS.2011.6127040.
- [2] V. Sreenath and B. George, "A novel closed-loop SC capacitance-to-frequency converter with high linearity," in 2017 IEEE International Instrumentation and Measurement Technology Conference (I2MTC), Turin, Italy, 2017, pp. 1-5, doi: 10.1109/I2MTC.2017.7969767.
- [3] ScioSense, "PCAP04 Capacitance-to-Digital Converter Datasheet," ScioSense GmbH, 2024. [Online].
Available: <https://www.sciosense.com/products/capacitance-to-digital-converters/pcap04/> (or use direct datasheet link if known)
- [4] I. R. Rodin, O. V. Korolkova, and I. N. Pavlov, "Rain and fog sensors based on frustrated total internal reflection," in 2025 7th International Youth Conference on Radio Electronics, Electrical and Power Engineering (REEPE), Moscow, Russian Federation, 2025, pp. 1-7, doi: 10.1109/REEPE63962.2025.10970791.
- [5] Nordic Semiconductor, "nRF52833 Product Specification v1.6," Nordic Semiconductor ASA, 2024. [Online].
Available: <https://infocenter.nordicsemi.com/>
- [6] Circuit Basics, "How to measure capacitance with an Arduino," . [Online].
Available: <https://circuitbasics.com/how-to-make-an-arduino-capacitance-meter/> (Accessed: 10 Mar. 2026).
- [7] D. Del Prete, F. Arcadio, C. Griffo, D. Cicatiello, L. Zeni, and N. Cennamo, "An Arduino-based plasmonic sensor to detect rain and its analysis," in 2022 IEEE International Symposium on Measurements & Networking (M&N), Padua, Italy, 2022, pp. 1-5, doi: 10.1109/MN55117.2022.9887648.

9. Appendix A

A.1 Core Capacitance Measurement Function (with Oversampling, Repeated Measurements, and Outlier Rejection)

```
// -----  
// Capacitance measurement core function (only small capacitance method, with  
enhanced stability)  
// -----  
static float measure_capacitance(void)  
{  
    uint32_t adc_vals[MEASURE_REPEAT];  
    uint32_t adc_sum;  
    nrf_saadc_value_t adc_val;  
    float capacitance;  
  
    // Perform multiple measurements for outlier rejection  
    for (int repeat = 0; repeat < MEASURE_REPEAT; repeat++) {  
        // ----- Forced discharge: ensure initial capacitor voltage zero  
        nrf_gpio_cfg_output(RAIN_SENSOR_OUT_PIN);  
        nrf_gpio_cfg_output(RAIN_SENSOR_IN_PIN);  
        nrf_gpio_pin_clear(RAIN_SENSOR_OUT_PIN);  
        nrf_gpio_pin_clear(RAIN_SENSOR_IN_PIN);  
        nrf_delay_ms(5);  
        // After discharge, restore to high impedance and wait for pins to stabilize  
        nrf_gpio_cfg_default(RAIN_SENSOR_OUT_PIN);  
        nrf_gpio_cfg_default(RAIN_SENSOR_IN_PIN);  
        nrf_delay_us(1000);          // extended stabilization time
```

```

// ----- Charge and read ADC -----
nrf_gpio_cfg_output(RAIN_SENSOR_OUT_PIN);
nrf_gpio_pin_set(RAIN_SENSOR_OUT_PIN);
nrf_delay_us(CHARGE_DELAY_US);      // increased charging time for
stability

nrf_gpio_cfg_default(RAIN_SENSOR_IN_PIN);
nrf_delay_us(10);      // wait for voltage to stabilize before ADC

// Multiple ADC reads and average to reduce noise (oversampling)
adc_sum = 0;
for (int i = 0; i < ADC_OVERSAMPLE; i++) {
    perform_saadc_sample(&adc_val,
RAIN_SENSOR_ADC_CHANNEL_IN);
    adc_sum += adc_val;
}
adc_vals[repeat] = adc_sum / ADC_OVERSAMPLE;  // store averaged
ADC value

// Discharge after each measurement
nrf_gpio_pin_clear(RAIN_SENSOR_OUT_PIN);
nrf_gpio_cfg_default(RAIN_SENSOR_OUT_PIN);

// Short delay between repetitions to allow full discharge
nrf_delay_ms(10);
}

// --- Outlier removal: sort the ADC values and discard lowest and highest
// Simple bubble sort for small array
for (int i = 0; i < MEASURE_REPEAT - 1; i++) {
    for (int j = i + 1; j < MEASURE_REPEAT; j++) {
        if (adc_vals[i] > adc_vals[j]) {
            uint32_t temp = adc_vals[i];

```

```

        adc_vals[i] = adc_vals[j];
        adc_vals[j] = temp;
    }
}
}

// Remove more outliers for better stability (trim 3 each side if enough samples)
int trim = (MEASURE_REPEAT >= 7) ? 3 : 1;
if (MEASURE_REPEAT <= 2*trim) trim = 0;
uint32_t sum = 0;
int count = MEASURE_REPEAT - 2*trim;
for (int i = trim; i < MEASURE_REPEAT - trim; i++) {
    sum += adc_vals[i];
}
uint32_t val = sum / count; // final averaged ADC value

// Calibration formula: net capacitance = val * K / (4095 - val) - C_stray
capacitance = (float)val * IN_STRAY_CAP_TO_GND /
(float)(MAX_ADC_VALUE - val) - C_STRAY;
if (capacitance < 0) capacitance = 0;
NRF_LOG_DEBUG("Small cap: ADC=%d, C=%.2f pF", val, capacitance);
return capacitance;
}

```

A.2 Moving Average Filter (Implemented in the RTC Interrupt Handler)

```
// -----  
// Moving average filter (5-point)  
// -----  
  
#define FILTER_SAMPLES      5  
static float g_cap_buffer[FILTER_SAMPLES] = {0};  
static uint8_t g_cap_idx = 0;  
static float g_cap_sum = 0;  
static uint8_t g_cap_cnt = 0;  
  
// Inside RTC handler:  
float raw_cap = measure_capacitance();  
if (raw_cap > 0.1f) {  
    g_cap_sum -= g_cap_buffer[g_cap_idx];  
    g_cap_buffer[g_cap_idx] = raw_cap;  
    g_cap_sum += raw_cap;  
    g_cap_idx = (g_cap_idx + 1) % FILTER_SAMPLES;  
    if (g_cap_cnt < FILTER_SAMPLES) g_cap_cnt++;  
    float avg_cap = g_cap_sum / g_cap_cnt;  
    process_rain_measurement(avg_cap);  
}
```

A.3 Rain Intensity Classification Function

```
// -----  
// Rain intensity classification  
// -----  
typedef enum {  
    RAIN_NONE = 0,  
    RAIN_LIGHT,  
    RAIN_MODERATE,  
    RAIN_HEAVY  
} rain_intensity_t;  
  
static rain_intensity_t classify_rain_intensity(float capacitance)  
{  
    // Thresholds can be adjusted according to the environment  
    if (capacitance < 150.0f)    return RAIN_NONE;  
    else if (capacitance < 250.0f) return RAIN_LIGHT;  
    else if (capacitance < 500.0f) return RAIN_MODERATE;  
    else                        return RAIN_HEAVY;  
}
```

A.4 Optional: Calibration Parameter Calculation

```
// Inside perform_calibration():  
// Calculate x = val / (4095 - val)  
float x1 = val1_avg / (MAX_ADC_VALUE - val1_avg);  
float x2 = val2_avg / (MAX_ADC_VALUE - val2_avg);  
  
// Solve equations to find K and C_stray  
float K = (C2 - C1) / (x2 - x1);  
float C_stray = x1 * K - C1; // or use x2*K - C2
```


10. Appendix B: Summary of Meeting Minutes

Date	Topics Discussed	Action Items / Outcomes
26 Sep 2025	Discussion of project core concept (low- cost capacitive rain sensor using MCU built- in ADC).	Search for relevant literature and organise into a report.
3 Oct 2025	Obtained nRF52833 DK development board; set up software environment.	Verify toolchain with a simple test program.
15 Oct 2025	Presentation of project understanding and task planning; demonstrated successful software environment with simple examples.	Begin porting Arduino capacitance meter algorithm to nRF52833.
17 Oct 2025	Emphasised core technical solution: using internal pull- up resistor and built- in ADC for direct capacitance measurement.	Study timing and GPIO configuration for small- capacitance method.
5 Nov 2025	Adapted SAADC driver code; resolved RTT debugging issue (header paths, library configuration).	Continue developing measurement loop; test with fixed capacitors.
13 Jan 2026	Discussed rain measurement code added based on reference code; reviewed initial ADC readings.	Improve measurement stability with oversampling and filtering.
27 Jan 2026	Received breadboard, jumper wires, and standard capacitors (10 pF, 100 pF) from supervisor. Clarified next phase: hardware connection and calibration.	Implement two- point calibration routine; verify K and C _{stray} .
17 Feb 2026	Demonstrated code with anti- interference processing (oversampling, outlier rejection, moving average). Showed stable static measurement of 10 pF and 100 pF.	Perform dynamic testing with actual rain sensor; define rain intensity thresholds.
20 Mar 2026	Presented final code meeting project objectives. Supervisor suggested cleaning up unnecessary code and improving readability.	Refactor code: remove large- capacitance branch, add comments, structure clearly. Prepare final report and appendix.